# Linear programming: solutions

## 29-E01 Free variables and minimization

### Derivation

Set $u=p-q$, $p, q \geq 0$, and maximize $-3p+3q-v$. Equality gives $p-q-v \leq 2$ and $-p+q+v \leq -2$; the other row is $p-q+v \leq 6$. Slacks are $s_1=2-p+q+v$, $s_2=-2+p-q-v$, $s_3=6-p+q-v$, with all $p$, $q$, $v$, $s$ nonnegative.

Original feasible $u$ maps to $p=\max(u,0)$, $q=\max(-u,0)$. A new feasible point maps back by $u=p-q$, preserving original equality/inequality and reversing the objective sign. Directly, $u=v+2$ and $2v+2 \leq 6$ give $0 \leq v \leq 2$. Objective $3(v+2)+v=6+4v$ is minimized at $v=0$, $u=2$, with value 6. The transformed maximum is $-6$, attained for example at $(p, q, v)=(2,0,0)$.

### Conclusion and checks

Negate the new maximum $-6$ to return original value 6. This example's optimal $u$ happens to be nonnegative; that does not justify imposing it generally. A feasible problem requiring $u=-1$ would become infeasible.

## 29-E02 Complete pivots and a dual certificate

### Derivation

Initially $x_3=10-2x_1-x_2$, $x_4=8-x_1-2x_2$, $z=4x_1+3x_2$. Entering $x_1$ has ratios 5,8, so $x_3$ leaves. New rows are $x_1=5-x_2/2-x_3/2$, $x_4=3-3x_2/2+x_3/2$, $z=20+x_2-2x_3$.

Enter $x_2$. Ratios from $x_1$, $x_4$ are $5/(1/2)=10$ and $3/(3/2)=2$, so $x_4$ leaves. The complete dictionary becomes

$$x_2=2+x_3/3-2x_4/3;\ x_1=4-2x_3/3+x_4/3;\ z=22-5x_3/3-2x_4/3.$$

Set $x_3=x_4=0$ to obtain(4,2). Constraints $8+2=10$ and $4+4=8$ are tight; objective $16+6=22$. Dual minimizes $10y_1+8y_2$ subject to $2y_1+y_2 \geq 4$, $y_1+2y_2 \geq 3$, $y \geq 0$. Slack reduced costs give $y=(5/3,2/3)$, both dual equalities, and value $50/3+16/3=22$.

### Conclusion and checks

Steps 5,2 lead to optimum(4,2), value 22. Both pivots retain leaving-variable coefficients. The same-value feasible dual independently certifies optimality.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Linear programming: solutions (continued)

## 29-E03 Zero ratios, ties, and symbolic perturbation

### Derivation

Entering $x_1$ gives ratios 0/1=0 for $x_3$ and 2/1=2 for $x_4$. Let $x_3$ leave at $\theta=0$. The dictionary becomes $x_1=-x_3$, $x_4=2+x_3-x_2$, $z=-x_3+x_2$. Decision variables remain 0 with $x_4=2$. Enter $x_2$:only $x_4$ has positive denominator, with $\theta=2$. New $x_2=2+x_3-x_4$, $x_1=-x_3$, $z=2-x_4$ has no positive reduced cost. At $x_3=x_4=0$, the optimum is(0,2), value 2. Direct feasibility also forces $x_1=0, 0\leq x_2\leq 2$.

For the symbolic example, ratios are $2+\varepsilon$ and $2+\varepsilon^2$. Their first difference is at $\varepsilon$ coefficient 1 versus 0, so$2+\varepsilon^2$ is smaller and $x_4$ leaves. For positive infinitesimal $\varepsilon$,$\varepsilon^2<\varepsilon$. Afterward $x_3=\varepsilon-\varepsilon^2+x_2+x_4$ has zero constant at nonbasics 0 but strictly positive symbolic value.

### Conclusion and checks

A zero step legally changes basis. The first example eventually moves, but its first degenerate pivot itself does not improve the point. Lexicographic comparison begins at low degree; an arbitrary floating $\varepsilon$ is not the exact rule.

## 29-E04 From Phase I through the original optimum

### Derivation

Original slacks $s_1=4-x$, $s_2=-2+x$ give $s_2=-2$ at x=0. Auxiliary rows are $s_1=4-x+a$, $s_2=-2+x+a$, $w=-a$. Special a-entry, $s_2$-exit gives $a=2-x+s_2$, $s_1=6-2x+s_2$, $w=-2+x-s_2$. Point(x, a, $s_1$, $s_2$)= (0,2,6,0) is now feasible.

Enter x:ratios 2 from a and 3 from $s_1$ make a leave. Obtain $x=2+s_2-a$, $s_1=2-s_2+2a$, $w=-a$, with x=2, a=0, $s_1=2$. Auxiliary optimum is 0. Delete a to get $x=2+s_2$, $s_1=2-s_2$. Restore $z=x=2+s_2$. Enter $s_2$ and let $s_1$ leave at $\theta=2$, obtaining $s_2=2-s_1$, $x=4-s_1$, $z=4-s_1$. Thus x=4 is optimal.

With bound $x\leq 1$, auxiliary initialization has $s_1=1-x+a$, $s_2=-2+x+a$. Special pivot gives $s_1=3-2x+s_2$, $a=2-x+s_2$, $w=-2+x-s_2$. Ratios 3/2 and 2 make $s_1$ leave when x enters. Final $x=3/2-s_1/2+s_2/2$, $a=1/2+s_1/2+s_2/2$, $w=-1/2-s_1/2-s_2/2$ proves auxiliary optimum−1/2 and original infeasibility.

### Conclusion and checks

Distinguish Phase I optimum 0 from original optimum 4. In the changed problem, $x\leq 1$ and$-x\leq -2$ sum to $0\leq -1$. Initial a=2 alone never justified an infeasibility conclusion.

# Linear programming: solutions (continued)

## 29-E05 Improving rays and unbounded regions

### Derivation

Initially $s=1-x_1+x_2$, $z=2x_1+x_2$. Enter $x_1$, leave $s$ at $\theta=1$, to get $x_1=1+x_2-s$, $z=2+3x_2-2s$. For entering $x_2$, the row's subtracted coefficient is $-1$ and imposes no bound. Set $s=0$, $x_2=\theta$, so $x=(1+\theta,\theta)$.

Choose $x^0=(1,0)$, $r=(1,1)$. Then $r\geq 0$, $A=(1,-1)$ gives $Ar=0\leq 0$, and $c^\top r=3>0$. Every $\theta\geq 0$ is feasible with objective $2+3\theta$, unbounded above. For objective $-x_1-x_2$, nonnegativity gives objective $\leq 0$ and the feasible origin attains 0.

### Conclusion and checks

Unboundedness of the region is geometric. Objective unboundedness depends on its positive change along a feasible ray, not merely on an infinitely extending drawing.

## 29-E06 Preconditions of the potential model

### Derivation

Full constraints are $d_s=0$, $d_a\leq -2$, $d_t-d_a\leq 3$, $d_t\leq 4$, $d_v-d_u\leq -1$, $d_u-d_v\leq -1$. The last two sum to $0\leq -2$, making the full LP infeasible because of the disconnected negative cycle.

Restrict to source-reachable$\{s, a, t\}$. The path s-a-t gives $d_t\leq 1$. Values $d_a=-2$, $d_t=1$ are feasible, so optimum 1 equals $\min(-2+3,4)$. Adding $d_a\geq 0$ contradicts $d_a\leq -2$ and excludes every correct potential.

### Conclusion and checks

Constraints have a scope. An unreachable component's negative cycle does not eliminate a finite s-to-t distance. The repaired potential labels may be negative.

## 29-E07 Shared capacities and integrality

### Derivation

Use branches s-a-t and s-b-t. For $k=1,2$, each arc has $g^k\geq 0$, conservation at a, b, and source/sink totals 1. Shared arc capacity is $g^1_e+g^2_e\leq 1$.

A fractional plan sends 1/2 of each commodity on each branch, total 1 per arc. An integral plan sends commodity 1 entirely above and commodity 2 entirely below, total 1 per arc again. Both satisfy the LP, but the first is not single-path per commodity. Allowing splitting does not require it.

# Linear programming: solutions (continued)

With one shared bottleneck of capacity 1 and demands 3/4 each, each commodity separately fits. Combined conservation forces total 3/2 through the bottleneck, violating capacity. Adding independent single-commodity solutions fails because they compete for the same capacity.

### Conclusion and checks

This instance having an integral solution does not make every multicommodity LP integral. Integer amounts and unsplittable paths are different extra constraints:an integer demand 2 may split into two paths carrying 1 each and still fail the single-path requirement.

## 29-E08 Complementary slackness and theorem boundaries

### Derivation

Dual is min $4y_1+2y_2$ with $y_1+y_2 \geq 2$, $y_1 \geq 1$, $y \geq 0$. Point $x=(2,2)$ satisfies $x_1+x_2=4$, $x_1=2$, with value 6. Point $y=(1,1)$ satisfies both dual equalities with value 4+2=6.

Primal slacks and dual slacks are both(0,0). Thus gap $y^T(b-Ax)+x^T(A^Ty-c)=0+0=0$, certifying both optima. For $x'=(1,3)$, primal slacks are(0,1), objective 5. Gap 1 with the same y is exactly $y_2(2-x_1)=1 \cdot 1$, violating complementarity.

With strict upper constraint $x<1$, every feasible x has a larger feasible$(x+1)/2<1$. No maximum exists, yet objective has finite upper bound 1. The problem is neither infeasible, nor upward-unbounded, nor finitely attained. This does not contradict the theorem, which assumes non-strict LP constraints.

### Conclusion and checks

A certificate checks feasibility, objective, and complementary products together. A supremum differs from a maximum attained by a point.

# Linear programming: solutions (continued)

## Verification code appendix: phase_one_certificate.py

```
"""Verify the Phase I states and a matching primal-dual certificate."""


def objective(x1, x2):
    return 2 * x1 + x2


def run():
    phase_rows = [
        (0.0, 0.0, 0.0, 5.0),
        (5.0, 0.0, 0.0, 0.0),
        (0.0, 5.0, 0.0, 0.0),
    ]
    assert all(
        abs(x1 + x2 - x3 + artificial - 5) < 1e-12
        and min(x1, x2, x3, artificial) >= 0
        for x1, x2, x3, artificial in phase_rows
    )
    assert objective(*phase_rows[1][:2]) == 10
    assert objective(*phase_rows[2][:2]) == 5

    primal = objective(0, 5)
    dual_y = 1
    dual_bound = 5 * dual_y
    assert primal == dual_bound == 5
    assert 1 * dual_y <= 2
    assert 1 * dual_y <= 1
    print("PASS phase-I artificial=5,w=0 primal=5 dual=5 ratio_rows_include_zero=True")
    return primal, dual_bound


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Linear programming: solutions (continued)

## Verification code appendix: simplex_exact.py

```
"""Exact two-phase dictionary simplex with symbolic lex RHS perturbation.

Fractions and polynomial coefficient tuples are a reference for mathematical
steps, not a production floating-point optimizer or a timing benchmark.
"""
from fractions import Fraction as F

class Dictionary:
    def __init__(self,B,N,b,a,c,v=0):
        self.B=list(B);self.N=list(N)
        self.b={i:F(b[i]) for i in B};self.a={(i,j):F(a.get((i,j),0)) for i in B for j in N}
        self.c={j:F(c.get(j,0)) for j in N};self.v=F(v);self.trace=[]
    def point(self):return {**{j:F(0) for j in self.N},**self.b}
    def pivot(self,l,e):
        p=self.a[l,e];assert p
        oldB=list(self.B);oldN=list(self.N);newN=[j for j in oldN if j!=e]+[l]
        newb={e:self.b[l]/p};newa={(e,l):1/p}
        for j in oldN:
            if j!=e:newa[e,j]=self.a[l,j]/p
        for i in oldB:
            if i==l:continue
            newb[i]=self.b[i]-self.a[i,e]*newb[e]
            newa[i,l]=-self.a[i,e]/p
            for j in oldN:
                if j!=e:newa[i,j]=self.a[i,j]-self.a[i,e]*newa[e,j]
        newv=self.v+self.c[e]*newb[e]
        newc={l:-self.c[e]/p}
        for j in oldN:
            if j!=e:newc[j]=self.c[j]-self.c[e]*newa[e,j]
        if hasattr(self,'lex'):
            old=self.lex;row=tuple(x/p for x in old[l]);self.lex={e:row}
            for i in oldB:
                if i!=l:self.lex[i]=tuple(x-self.a[i,e]*y for x,y in zip(old[i],row))
        self.B=[e if i==l else i for i in oldB];self.N=newN
        self.b=newb;self.a=newa;self.c=newc;self.v=newv

self.trace.append({'enter':e,'leave':l,'b':dict(self.b),'a':dict(self.a),'c':dict(self.c),'v'
:self.v})
    def optimize(self):
        assert all(b>=0 for b in self.b.values())
        m=len(self.B)
        self.lex={i:(self.b[i],)+tuple(F(int(k==r)) for k in range(m)) for r,i in
enumerate(self.B)}
        visited=set()
        while True:
            state=frozenset(self.B);assert state not in visited;visited.add(state)
            choices=[j for j in self.N if self.c[j]>0]
            if not choices:del self.lex;return 'optimal',None
            e=min(choices);rows=[i for i in self.B if self.a[i,e]>0]
            if not rows:
                ray={j:F(0) for j in self.N};ray[e]=F(1)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Linear programming: solutions (continued)

```
                ray.update({i:-self.a[i,e] for i in self.B});del self.lex;return
'unbounded',ray
            l=min(rows,key=lambda i:tuple(x/self.a[i,e] for x in self.lex[i]))
            self.pivot(l,e)
            assert all(b>=0 for b in self.b.values())
            assert all(next((x for x in row if x),0)>0 for row in self.lex.values())

def solve(A,b,c):
    A=[[F(x) for x in row] for row in A];b=list(map(F,b));c=list(map(F,c))
    m=len(b);n=len(c);B=list(range(n,n+m));N=list(range(n))
    d=Dictionary(B,N,dict(zip(B,b)),{(n+i,j):A[i][j] for i in range(m) for j in
N},dict(enumerate(c)))
    phase1=[]
    if m and min(b)<0:
        artificial=-1;d.N.append(artificial);d.c={j:F(0) for j in d.N};d.c[artificial]=F(-1)
        for i in d.B:d.a[i,artificial]=F(-1)
        l=min(d.B,key=lambda i:d.b[i]);d.pivot(l,artificial)
        status,_=d.optimize();assert status=='optimal';phase1=list(d.trace)
        if d.v<0:return {'status':'infeasible','phase1':phase1,'auxiliary_value':d.v}
        assert d.v==0
        if artificial in d.B:
            rows=[j for j in d.N if d.a[artificial,j]]
            if rows:d.pivot(artificial,min(rows))
            else:
                d.B.remove(artificial);d.b.pop(artificial)
                d.a={ij:x for ij,x in d.a.items() if ij[0]!=artificial}
        if artificial in d.N:d.N.remove(artificial)
        d.a={ij:x for ij,x in d.a.items() if artificial not in ij}
        d.v=sum(c[i]*d.b[i] for i in d.B if 0<=i<n)
        d.c={j:(c[j] if 0<=j<n else F(0))-sum(c[i]*d.a[i,j] for i in d.B if 0<=i<n) for j in
d.N}
        d.trace=[]
    status,ray=d.optimize();q=d.point();x=[q.get(j,F(0)) for j in range(n)]
    assert all(sum(a*t for a,t in zip(row,x))<=rhs for row,rhs in zip(A,b))
    if status=='unbounded':
        r=[ray.get(j,F(0)) for j in range(n)]
        assert all(v>=0 for v in r) and all(sum(a*t for a,t in zip(row,r))<=0 for row in A)
        assert sum(a*t for a,t in zip(c,r))>0
        return {'status':status,'x':x,'ray':r,'phase1':phase1,'phase2':d.trace}
    value=sum(a*t for a,t in zip(c,x));assert value==d.v
    y=[-d.c.get(n+i,F(0)) for i in range(m)]
    assert all(v>=0 for v in y)
    assert all(sum(A[i][j]*y[i] for i in range(m))>=c[j] for j in range(n))
    assert sum(b[i]*y[i] for i in range(m))==value
    return {'status':status,'x':x,'value':value,'dual':y,'phase1':phase1,'phase2':d.trace}

if __name__=='__main__':
    for args in [([[1,2],[3,1]],[8,9],[3,2]),([[-1,-1]],[-5],[-2,-1]),([[1],[-1]],[1,-2],
[1]),([[-1]],[-1],[1])]:
        print(solve(*args))
```

# Linear programming: solutions (continued)

### References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study